

```

import React, { useState, useEffect } from 'react';
import {
  FileText, Calendar, DollarSign, Users, Activity, Pill,
  Stethoscope, Download, Building2, AlertCircle, Bed,
  ChevronRight, ClipboardList, X, RefreshCw
} from 'lucide-react';
import api from '../utils/api';
import jsPDF from 'jspdf';
import 'jspdf-autotable';

/* -----
  Helpers
----- */
const fmt = (n) => Number(n || 0).toLocaleString();
const fmtTZS = (n) => `TZS ${fmt(n)}`;
const fmtDate = (d) => {
  if (!d) return '--';
  try { return new Date(d).toLocaleDateString('en-GB', { day: '2-digit', month: '2-digit' }); }
  catch { return '--'; }
};

const ageGroup = (dob) => {
  if (!dob) return '5To60Years';
  const months = Math.floor((Date.now() - new Date(dob).getTime()) / (1000 * 60 * 60 * 24 * 30));
  if (months < 1) return 'under1Month';
  if (months < 12) return '1MonthTo1Year';
  if (months < 60) return '1YearTo5Years';
  if (months < 720) return '5To60Years';
  return 'over60Years';
};

/* -----
  Stat Card
----- */
const StatCard = ({ icon: Icon, label, value, sub, color, badge }) => (
  <div className={`rounded-xl p-5 text-white shadow-md ${color} flex flex-col gap-4`} >
    <div className="flex items-start justify-between">
      <div className="bg-white/20 rounded-lg p-2"><Icon className="w-5 h-5" /></div>
      {badge && <span className="text-xs font-semibold bg-white/20 px-2 py-0.5 rounded">{badge}</span>}
    </div>
    <div>
      <div className="text-2xl font-bold tracking-tight leading-none">{value}</div>
      <div className="text-sm font-medium mt-1 opacity-90">{label}</div>
      {sub && <div className="text-xs mt-0.5 opacity-70">{sub}</div>}
    </div>
  </div>
);

```

```

    </div>
  </div>
);

/* -----
   Report Table
----- */
const ReportTable = ({ report, data }) => {
  const rows = (() => {
    if (Array.isArray(data)) return data;
    if (data?.data?.invoices && Array.isArray(data.data.invoices)) return data.data.invoices;
    if (data?.data?.payments && Array.isArray(data.data.payments)) return data.data.payments;
    if (data?.data && Array.isArray(data.data)) return data.data;
    return [];
  })();

  if (rows.length === 0) {
    return (
      <div className="flex flex-col items-center justify-center py-16 text-gray-400">
        <ClipboardList className="w-12 h-12 mb-3 opacity-30" />
        <p className="text-sm">No records found for the selected period.</p>
      </div>
    );
  }

  const columns = (() => {
    const id = report.id;
    if (['patient-visits', 'all-patient-visits', 'visits-summary'].includes(id)) return [
      { label: 'Visit ID', get: r => r.visitId || r._id?.slice(-6)?.toUpperCase() },
      { label: 'Patient', get: r => `${r.patient?.firstName} ${r.patient?.lastName}` },
      { label: 'Doctor', get: r => `${r.doctor?.firstName} ${r.doctor?.lastName}` },
      { label: 'Date', get: r => fmtDate(r.visitDate || r.createdAt) },
      { label: 'Type', get: r => r.type || '--' },
      { label: 'Status', get: r => r.status || '--', badge: true },
    ];
    if (id.startsWith('invoice')) return [
      { label: 'Invoice #', get: r => r.invoiceNumber || '--' },
      { label: 'Patient', get: r => `${r.patient?.firstName} ${r.patient?.lastName}` },
      { label: 'Total', get: r => fmtTZS(r.totalAmount) },
      { label: 'Paid', get: r => fmtTZS(r.amountPaid) },
      { label: 'Balance', get: r => fmtTZS(r.balanceDue) },
      { label: 'Status', get: r => r.status || '--', badge: true },
      { label: 'Date', get: r => fmtDate(r.createdAt) },
    ];
    if (id.startsWith('ipd')) return [
      { label: 'Admission #', get: r => r.admissionNumber || r._id?.slice(-6)?.toUpperCase() },
      { label: 'Patient', get: r => `${r.patient?.firstName} ${r.patient?.lastName}` },
    ];
  });

```

```

    { label: 'Ward',          get: r => r.ward?.name || '--' },
    { label: 'Bed',          get: r => r.bed?.bedNumber || '--' },
    { label: 'Type',         get: r => r.admissionType || '--' },
    { label: 'Admitted',     get: r => fmtDate(r.admissionDate) },
    { label: 'Discharged',   get: r => fmtDate(r.dischargeDate) || '--' },
    { label: 'Status',       get: r => r.status || '--', badge: true },
  ];
  if (id === 'prescriptions') return [
    { label: 'Medicine',     get: r => r.medication || '--' },
    { label: 'Patient',     get: r => `${r.patient?.firstName||''} ${r.patient?.
    { label: 'Dosage',       get: r => r.dosage || '--' },
    { label: 'Frequency',   get: r => r.frequency || '--' },
    { label: 'Status',       get: r => r.status || '--', badge: true },
    { label: 'Date',        get: r => fmtDate(r.createdAt) },
  ];
  if (id === 'lab-tests') return [
    { label: 'Test',        get: r => r.testName || '--' },
    { label: 'Patient',     get: r => `${r.patient?.firstName||''} ${r.patient?.
    { label: 'Results',     get: r => r.results || 'Pending' },
    { label: 'Status',       get: r => r.status || '--', badge: true },
    { label: 'Date',        get: r => fmtDate(r.createdAt) },
  ];
  if (id === 'radiology') return [
    { label: 'Scan Type',   get: r => r.scanType || '--' },
    { label: 'Body Part',   get: r => r.bodyPart || '--' },
    { label: 'Patient',     get: r => `${r.patient?.firstName||''} ${r.patient?.
    { label: 'Status',       get: r => r.status || '--', badge: true },
    { label: 'Date',        get: r => fmtDate(r.createdAt) },
  ];
  if (id.startsWith('procedures')) return [
    { label: 'Procedure',   get: r => r.procedure_name || r.procedureName || '--
    { label: 'Patient',     get: r => `${r.patient?.firstName||''} ${r.patient?.
    { label: 'Surgeon',     get: r => `${r.surgeon?.firstName||''} ${r.surgeon?.
    { label: 'Theatre',     get: r => r.theatre?.name || '--' },
    { label: 'Date',        get: r => fmtDate(r.procedure_date || r.procedureDat
    { label: 'Status',       get: r => r.status || '--', badge: true },
  ];
  if (id === 'dispensing-records') return [
    { label: 'Medicine',     get: r => r.medication || r.medicineName || r.medici
    { label: 'Patient',     get: r => `${r.patient?.firstName||''} ${r.patient?.
    { label: 'Qty',          get: r => r.quantityDispensed || r.quantity || '--'
    { label: 'Pharmacist',  get: r => `${r.pharmacist?.firstName||''} ${r.pharma
    { label: 'Date',        get: r => fmtDate(r.dispensedDate || r.createdAt) },
  ];
  if (id === 'direct-dispensing') return [
    { label: 'Medicine',     get: r => r.medicineName || r.medication || '--' },
    { label: 'Qty',          get: r => r.quantity || '--' },

```

```

    { label: 'Amount',      get: r => fmtTZS(r.totalAmount || r.amount) },
    { label: 'Date',        get: r => fmtDate(r.createdAt) },
  ];
  if (id === 'medicines-inventory') return [
    { label: 'Name',        get: r => r.name || '--' },
    { label: 'Category',    get: r => r.category || '--' },
    { label: 'Stock',       get: r => fmt(r.quantityInStock) },
    { label: 'Unit',        get: r => r.unit || '--' },
    { label: 'Price',       get: r => fmtTZS(r.sellingPrice) },
  ];
  const keys = Object.keys(rows[0] || {}).filter(k => typeof rows[0][k] !== 'object');
  return keys.map(k => ({ label: k, get: r => String(r[k] ?? '--') }));
}());

const statusColor = (s = '') => {
  const l = s.toLowerCase();
  if (['paid', 'completed', 'discharged', 'active', 'dispensed', 'in queue'].includes(l)) return 'bg-white';
  if (['pending', 'scheduled', 'admitted', 'stable', 'pending payment', 'pending qua'].includes(l)) return 'bg-gray-100';
  if (['overdue', 'cancelled', 'deceased', 'critical'].includes(l)) return 'bg-red';
  return 'bg-gray-100 text-gray-600';
};

return (
  <div className="overflow-x-auto rounded-lg border border-gray-200">
    <table className="min-w-full divide-y divide-gray-100 text-sm">
      <thead className="bg-gray-50">
        <tr>
          {columns.map(col => (
            <th key={col.label} className="px-4 py-3 text-left text-xs font-semi-bold">
              {col.label}
            </th>
          ))}
        </tr>
      </thead>
      <tbody className="divide-y divide-gray-100 bg-white">
        {rows.slice(0, 200).map((row, i) => (
          <tr key={i} className="hover:bg-gray-50 transition-colors">
            {columns.map(col => (
              <td key={col.label} className="px-4 py-3 text-gray-700 whitespace-normal">
                {col.badge
                  ? <span className="px-2 py-0.5 rounded-full text-xs font-med">
                      {col.get(row)}
                    </span>
                  : col.get(row)}
            ))}
          </td>
        ))}
      </tr>
    </tbody>
  </table>
);

```

```

        </tbody>
      </table>
      {rows.length > 200 && (
        <div className="px-4 py-2 text-xs text-gray-400 bg-gray-50 border-t">
          Showing first 200 of {fmt(rows.length)} records. Export for full data.
        </div>
      )}
    </div>
  );
};

/* -----
   HMISSummary Views
   ----- */
const HMISSummary = ({ data, reportId }) => {
  if (!data?.data) return null;
  const d = data.data;

  if (reportId === 'hmis-opd') return (
    <div className="space-y-5">
      <div className="grid grid-cols-3 gap-4">
        {[{label: 'Total Visits', value: fmt(d.totalVisits), color: 'blue'}, {label: 'Ne
          <div key={s.label} className={`bg-${s.color}-50 rounded-xl p-4 text-cen
            <div className={`text-3xl font-bold text-${s.color}-700`} >{s.value}</
            <div className={`text-xs text-${s.color}-500 mt-1`} >{s.label}</div>
          </div>
        ]]}
      </div>
      <div className="grid grid-cols-3 gap-4">
        {[{label: 'Insurance', value: fmt(d.byPaymentMethod?.insurance), color: 'green
          <div key={s.label} className={`bg-${s.color}-50 rounded-xl p-4 text-cen
            <div className={`text-2xl font-bold text-${s.color}-700`} >{s.value}</
            <div className={`text-xs text-${s.color}-500 mt-1`} >{s.label}</div>
          </div>
        ]]}
      </div>
      {Object.keys(d.byDiagnosis||{}).length > 0 && (
        <div>
          <h4 className="text-sm font-semibold text-gray-600 mb-2">Top Diagnoses<
          <div className="rounded-lg border border-gray-200 overflow-hidden">
            <table className="min-w-full text-sm">
              <thead className="bg-gray-50"><tr>{['Diagnosis', 'Male', 'Female', 'To
              <tbody className="divide-y divide-gray-100">
                {Object.entries(d.byDiagnosis).sort((a,b)=>b[1].count-a[1].count)
                <tr key={cond} className="hover:bg-gray-50">
                  <td className="px-4 py-2 text-gray-700">{cond}</td>
                  <td className="px-4 py-2 text-center text-gray-600">{fmt(c.ma

```

```

        <td className="px-4 py-2 text-center text-gray-600">{fmt(c.fe
        <td className="px-4 py-2 text-center font-semibold">{fmt(c.co
    </tr>
    )})
  </tbody>
</table>
</div>
</div>
  )}
</div>
);

if (reportId === 'hmis-ipd') return (
  <div className="space-y-5">
    <div className="grid grid-cols-3 gap-4">
      {[{label: 'Total Admissions', value: fmt(d.totalAdmissions), color: 'violet'},
      <div key={s.label} className={`bg-${s.color}-50 rounded-xl p-4 text-cen
        <div className={`text-3xl font-bold text-${s.color}-700`} >{s.value}</
        <div className={`text-xs text-${s.color}-500 mt-1`} >{s.label}</div>
      </div>
    )]}
  </div>
  {Object.keys(d.byStatus||{}).length > 0 && (
    <div className="grid grid-cols-2 gap-2">
      {Object.entries(d.byStatus).map(([status, count])=>(
        <div key={status} className="flex items-center justify-between bg-gra
          <span className="capitalize text-gray-600">{status.replace('_', ' ')}
          <span className="font-semibold text-gray-800">{fmt(count)}</span>
        </div>
      )]}
    </div>
  )}
  {Object.keys(d.byDiagnosis||{}).length > 0 && (
    <div>
      <h4 className="text-sm font-semibold text-gray-600 mb-2">Diagnoses</h4>
      <div className="rounded-lg border border-gray-200 overflow-hidden">
        <table className="min-w-full text-sm">
          <thead className="bg-gray-50"><tr>{['Condition', 'Male', 'Female', 'To
          <tbody className="divide-y divide-gray-100">
            {Object.entries(d.byDiagnosis).sort((a,b)=>b[1].count-a[1].count)}
            <tr key={cond} className="hover:bg-gray-50">
              <td className="px-4 py-2 text-gray-700">{cond}</td>
              <td className="px-4 py-2 text-center text-gray-600">{fmt(c.ma
              <td className="px-4 py-2 text-center text-gray-600">{fmt(c.fe
              <td className="px-4 py-2 text-center font-semibold">{fmt(c.co
            </tr>
          )})
        </tbody>
      </table>
    </div>
  )}
);

```

```

        </tbody>
      </table>
    </div>
  </div>
  })
</div>
);

if (reportId === 'hmis-bed-occupancy') return (
  <div className="space-y-3">
    {[{label: 'Currently Occupied Beds', value: fmt(d.occupiedBeds), color: 'blue'},
      <div key={s.label} className={`flex items-center justify-between bg-${s.c}
        <span className={`text-sm text-${s.color}-600`} >{s.label}</span>
        <span className={`text-xl font-bold text-${s.color}-700`} >{s.value}</span>
      </div>
    ]}
    <p className="text-xs text-gray-400 pt-1">Total bed capacity requires fetch
  </div>
);

if (reportId === 'hmis-dtc') return (
  <div className="rounded-lg border border-gray-200 overflow-hidden">
    <table className="min-w-full text-sm">
      <thead className="bg-gray-50"><tr><th className="px-4 py-2 text-left text
      <tbody className="divide-y divide-gray-100">
        {[['Total DTC Cases', d.totalCases], ['Severe Dehydration', d.severeDehydr
          <tr key={label} className="hover:bg-gray-50">
            <td className="px-4 py-2.5 text-gray-700">{label}</td>
            <td className="px-4 py-2.5 text-center font-semibold text-gray-800"
          </tr>
        ]]}
      </tbody>
    </table>
  </div>
);

if (reportId === 'hmis-tracer-medicine') return (
  <div className="rounded-lg border border-gray-200 overflow-hidden">
    <table className="min-w-full text-sm">
      <thead className="bg-gray-50"><tr>{['#', 'Medicine', 'Service?', 'Available?
      <tbody className="divide-y divide-gray-100">
        {(d.tracerMedicines||[]).map((item,i)=>(
          <tr key={i} className="hover:bg-gray-50">
            <td className="px-4 py-2 text-gray-400 text-xs text-center">{i+1}</td>
            <td className="px-4 py-2 text-gray-700">{item.medicine}</td>
            <td className="px-4 py-2 text-center"><span className={`px-2 py-0.5
            <td className="px-4 py-2 text-center"><span className={`px-2 py-0.5

```

```

        <td className="px-4 py-2 text-center text-xs text-gray-600">{item.s
    </tr>
    )}}
  </tbody>
</table>
</div>
);
return null;
};

/* -----
Billing Statistics Summary
Uses confirmed fields: invoices.{totalInvoices,totalAmount,totalPaid,totalDue}
----- */
const BillingStatsSummary = ({ data }) => {
  const inv = data?.data?.invoices || {};
  const payments = data?.data?.payments || [];
  const overdueCount = data?.data?.overdueCount || 0;
  return (
    <div className="space-y-5">
      <div className="grid grid-cols-2 lg:grid-cols-4 gap-4">
        [{label: 'Total Invoices', value: fmt(inv.totalInvoices), color: 'blue'}, {lab
          <div key={s.label} className={`bg-${s.color}-50 rounded-xl p-4 text-cen
            <div className={`text-xl font-bold text-${s.color}-700`} >{s.value}</d
            <div className={`text-xs text-${s.color}-500 mt-1`} >{s.label}</div>
          </div>
        )}]
      </div>
      <div className="flex items-center gap-3 bg-red-50 rounded-xl px-5 py-3">
        <AlertCircle className="w-4 h-4 text-red-500" />
        <span className="text-sm text-red-700">{fmt(overdueCount)} overdue invoic
      </div>
      {payments.length > 0 && (
        <div>
          <h4 className="text-sm font-semibold text-gray-600 mb-2">Collections by
          <div className="rounded-lg border border-gray-200 overflow-hidden">
            <table className="min-w-full text-sm">
              <thead className="bg-gray-50"><tr><th className="px-4 py-2 text-lef
              <tbody className="divide-y divide-gray-100">
                {payments.map(p=>(
                  <tr key={p._id} className="hover:bg-gray-50">
                    <td className="px-4 py-2.5 capitalize text-gray-700">{(p._id|
                    <td className="px-4 py-2.5 text-center text-gray-600">{fmt(p.
                    <td className="px-4 py-2.5 text-right font-semibold">{fmtTZS(
                  </tr>
                )}}
              </tbody>

```



```

    </table>
  </div>
</div>
  })
</div>
);
};

/* -----
  IPD Statistics Summary
  Uses confirmed fields: currentAdmissions, statusCounts[[_id,count]]
  ----- */
const IPDStatsSummary = ({ data }) => {
  const d = data?.data || {};
  return (
    <div className="space-y-4">
      <div className="grid grid-cols-2 lg:grid-cols-3 gap-4">
        {[{label: 'Current Admissions', value: fmt(d.currentAdmissions), color: 'violet'}]
          <div key={s.label} className={`bg-${s.color}-50 rounded-xl p-4 text-center`}
            <div className={`text-3xl font-bold text-${s.color}-700`} >{s.value}</div>
            <div className={`text-xs text-${s.color}-500 mt-1`} >{s.label}</div>
          </div>
        )]}
      </div>
      {Array.isArray(d.statusCounts) && d.statusCounts.length > 0 && (
        <div className="grid grid-cols-2 gap-2">
          {d.statusCounts.map(s=>(
            <div key={s._id||s.status} className="flex items-center justify-between">
              <span className="capitalize text-gray-600">{(s._id||s.status||'').replace(/_/g, ' ')}</span>
              <span className="font-semibold text-gray-800">{fmt(s.count)}</span>
            </div>
          ))}
        </div>
      )}
    </div>
  );
};

/* -----
  PDF Export
  ----- */
const buildPDF = (rawData, report, dateRange, facilityName) => {
  const doc = new jsPDF();
  const pw = doc.internal.pageSize.getWidth();
  doc.setFontSize(16); doc.setFont(undefined, 'bold');
  doc.text(report.name, 14, 20);
  doc.setFontSize(9); doc.setFont(undefined, 'normal');

```

```

doc.text(`Facility: ${facilityName}`, 14, 28);
doc.text(`Generated: ${new Date().toLocaleString()}`, 14, 34);
if (dateRange.start) doc.text(`Period: ${dateRange.start} -> ${dateRange.end}||'

const rows = Array.isArray(rawData) ? rawData
  : rawData?.data?.invoices ? rawData.data.invoices
  : rawData?.data && Array.isArray(rawData.data) ? rawData.data
  : [];

let head=[], body=[];
const id = report.id;
if ([ 'patient-visits', 'all-patient-visits', 'visits-summary' ].includes(id)) {
  head=[ 'Visit ID', 'Patient', 'Doctor', 'Date', 'Type', 'Status' ];
  body=rows.map(r=>[r.visitId||'',`${r.patient?.firstName}||''`] ${r.patient?.las
} else if (id.startsWith('invoice')) {
  head=[ 'Invoice #', 'Patient', 'Total', 'Paid', 'Balance', 'Status' ];
  body=rows.map(r=>[r.invoiceNumber||'',`${r.patient?.firstName}||''`] ${r.pati
} else if (id.startsWith('ipd')) {
  head=[ 'Admission #', 'Patient', 'Ward', 'Bed', 'Admitted', 'Status' ];
  body=rows.map(r=>[r.admissionNumber||'',`${r.patient?.firstName}||''`] ${r.pati
} else if (id==='medicines-inventory') {
  head=[ 'Name', 'Category', 'Stock', 'Unit', 'Price' ];
  body=rows.map(r=>[r.name||'',r.category||'',fmt(r.quantityInStock),r.unit||''
} else {
  const keys=rows[0]?Object.keys(rows[0]).filter(k=>typeof rows[0][k]!='object
  head=keys; body=rows.map(r=>keys.map(k=>String(r[k]??'')));
}

if (body.length) {
  doc.autoTable({ startY: dateRange.start?46:40, head:[head], body, theme:'stri
} else {
  doc.setFontSize(11); doc.text('No records found.',14,55);
}

const pages=doc.internal.getNumberOfPages();
for(let i=1;i<=pages;i++){doc.setPage(i);doc.setFontSize(7.5);doc.text(`Page ${
return doc;
};

/* -----
  HMIS Processors
----- */

const processOPDData = (rawData) => {
  const visits = rawData?.data||(Array.isArray(rawData)?rawData:[]);
  const empty={()=>({male:0,female:0,total:0})};
  const byAgeGroup={under1Month:empty(),'1MonthTo1Year':empty(),'1YearTo5Years':e
  const byDiagnosis={}, byPaymentMethod={insurance:0,cash:0,waiver:0};
  let newPatients=0, repeatVisits=0;

```

```

visits.forEach(v=>{
  const t=(v.type||'').toLowerCase();
  if(t==='new' || t==='new patient') newPatients++; else repeatVisits++;
  const p=v.patient||{};
  const gender=(p.gender||'').toLowerCase()==='female'?'female':'male';
  const ag=ageGroup(p.dateOfBirth||p.dob);
  if(byAgeGroup[ag]){byAgeGroup[ag][gender]++;byAgeGroup[ag].total++;}
  (v.diagnosis||[]).forEach(d=>{
    const c=d.condition||'Unknown';
    if(!byDiagnosis[c]) byDiagnosis[c]={count:0,male:0,female:0};
    byDiagnosis[c].count++; byDiagnosis[c][gender]++;
  });
  if(p.insurance?.provider) byPaymentMethod.insurance++;
  else if(v.paymentMethod==='waiver') byPaymentMethod.waiver++;
  else byPaymentMethod.cash++;
});
return {totalVisits:visits.length,newPatients,repeatVisits,byAgeGroup,byDiagnosis};
};

```

```

const processIPDData = (rawData) => {
  const records=rawData?.data||(Array.isArray(rawData)?rawData:[]);
  const empty={()=>({male:0,female:0,total:0})};
  const byAgeGroup={under1Month:empty(),'1MonthTo1Year':empty(),'1YearTo5Years':empty()};
  const byDiagnosis={}, byStatus={};
  let deaths=0, totalDays=0;
  records.forEach(r=>{
    const p=r.patient||{};
    const gender=(p.gender||'').toLowerCase()==='female'?'female':'male';
    const ag=ageGroup(p.dateOfBirth);
    if(byAgeGroup[ag]){byAgeGroup[ag][gender]++;byAgeGroup[ag].total++;}
    const status=r.status||'unknown';
    byStatus[status]=(byStatus[status]||0)+1;
    if(r.dischargeReason==='deceased' || status==='deceased') deaths++;
    (r.diagnosis||[]).forEach(d=>{
      const c=d.condition||'Unknown';
      if(!byDiagnosis[c]) byDiagnosis[c]={count:0,male:0,female:0};
      byDiagnosis[c].count++; byDiagnosis[c][gender]++;
    });
    if(r.admissionDate){
      const end=r.dischargeDate?new Date(r.dischargeDate):new Date();
      totalDays+=Math.max(0,Math.ceil((end-new Date(r.admissionDate))/86400000));
    }
  });
  return {totalAdmissions:records.length,byAgeGroup,byDiagnosis,byStatus,averageLengthOfStay};
};

```

```

const processBedOccupancyData = (rawData) => {

```

```

const records=rawData?.data|| (Array.isArray(rawData)?rawData:[]);
const admitted=records.filter(r=>['admitted','under_observation','critical','st
return {occupiedBeds:admitted.length,maternityWard:{admitted:admitted.filter(r=
});

const processDTCDData = (rawData) => {
  const visits=rawData?.data|| (Array.isArray(rawData)?rawData:[]);
  const dtc=visits.filter(v=>(v.diagnosis|| []).some(d=>/diarr|dysentery/i.test(d.
  return {totalCases:dtc.length,severeDehydration:0,someDehydration:0,bloodInStoo
});

const processTracerMedicineData = (rawData) => {
  const medicines=rawData?.data|| (Array.isArray(rawData)?rawData:[]);
  const list=['DPT+HepB/HiB vaccine','Vitamin A','ALU 20/120mg','Sulphadoxine Pyr
  return {tracerMedicines:list.map(name=>{
    const found=medicines.find(m=>m.name?.toLowerCase().includes(name.toLowerCase
    const qty=found?.quantityInStock||0;
    return {medicine:name,serviceProvided:found?'Yes':'No',available:found&&qty>0
  })});
};

/* -----
  Main Component
----- */
const Reports = () => {
  const [selectedReport, setSelectedReport] = useState(null);
  const [dateRange, setDateRange] = useState({ start: '', end: '' });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [reportData, setReportData] = useState(null);
  const [facilityName, setFacilityName] = useState('Segese Medical Clinic');
  const [stats, setStats] = useState({ activeVisits:0, totalRevenue:0, pendingAmo

  useEffect(() => { fetchStatistics(); }, []);

  const fetchStatistics = async () => {
    try {
      const [billingRes, ipdRes, visitsRes] = await Promise.allSettled([
        api.get('/billing/statistics'),
        api.get('/ipd-records/statistics'),
        api.get('/visits'),
      ]);
      // /api/billing/statistics -> { data: { invoices: { totalPaid, totalDue },
      const billing = billingRes.status==='fulfilled' ? billingRes.value?.data?.d
      // /api/ipd-records/statistics -> { data: { currentAdmissions } }
      const ipd = ipdRes.status==='fulfilled' ? ipdRes.value?.data?.data : null;
      // /api/visits -> { data: [...] } only isActive:true

```

```

const visits = visitsRes.status==='fulfilled' ? visitsRes.value?.data?.data
setStats({
  activeVisits:      Array.isArray(visits) ? visits.length : 0,
  totalRevenue:      billing?.invoices?.totalPaid || 0,
  pendingAmount:      billing?.invoices?.totalDue  || 0,
  overdueCount:      billing?.overdueCount        || 0,
  currentAdmissions: ipd?.currentAdmissions       || 0,
});
} catch { /* fail silently */ }
};

const reportCategories = [
  { id:'hmis', title:'HMIS Reports', icon:FileText, color:'from-indigo-600 to-i
    // isActive=all -> backend returns full historical dataset, not just active
    { id:'hmis-opd',          name:'OPD Monthly Report',          description:'
    { id:'hmis-ipd',          name:'IPD Monthly Report',          description:'
    { id:'hmis-bed-occupancy', name:'Bed Occupancy Report',      description:'
    { id:'hmis-dtc',          name:'DTC Report',                  description:'
    { id:'hmis-tracer-medicine', name:'Tracer Medicine Report',  description:'
  ]],
  { id:'patient', title:'Patient Reports', icon:Users, color:'from-blue-600 to-
    { id:'patient-visits',      name:'Active Patient Visits',      description:'C
    { id:'all-patient-visits',  name:'All Patient Visits',          description:'F
  ]],
  { id:'clinical', title:'Clinical Reports', icon:Stethoscope, color:'from-teal
    { id:'visits-summary',      name:'Visits Summary',            description:'Full visit hi
    { id:'prescriptions',       name:'Prescriptions',            description:'All prescript
    { id:'lab-tests',           name:'Lab Tests',                  description:'Lab test orde
    { id:'radiology',           name:'Radiology',                  description:'Imaging order
  ]],
  { id:'ipd', title:'IPD Reports', icon:Bed, color:'from-violet-600 to-violet-7
    { id:'ipd-admissions',      name:'All Admissions',            description:'Complete
    { id:'ipd-statistics',      name:'IPD Statistics',            description:'Admission
    { id:'ipd-active',          name:'Active Admissions',         description:'Currently
    { id:'ipd-discharged',      name:'Discharged Patients',        description:'Discharge
  ]],
  { id:'financial', title:'Financial Reports', icon:DollarSign, color:'from-eme
    { id:'invoice-list',        name:'All Invoices',              description:'Complete
    { id:'invoice-pending',     name:'Pending Invoices',          description:'Unpaid i
    { id:'invoice-paid',        name:'Paid Invoices',              description:'Complete
    { id:'billing-statistics',  name:'Billing Statistics',          description:'Revenue
  ]],
  { id:'pharmacy', title:'Pharmacy Reports', icon:Pill, color:'from-rose-600 to
    { id:'dispensing-records',  name:'Dispensing Records',        description:'Patien
    { id:'direct-dispensing',   name:'Direct Dispensing',         description:'Over-t
    { id:'medicines-inventory', name:'Medicines Inventory',        description:'Stock
  ]],

```

```

    { id:'theatre', title:'Theatre Reports', icon:Activity, color:'from-orange-60'
    { id:'procedures-list',      name:'All Procedures',      description:'Comp
    { id:'procedures-scheduled', name:'Scheduled Procedures', description:'Upco
    { id:'procedures-completed', name:'Completed Procedures', description:'Fini
  ]},
];

```

```

const handleGenerateReport = async (report) => {
  setSelectedReport(report);
  setLoading(true);
  setError(null);
  setReportData(null);
  try {
    const params = {...(report.params||{}), limit:1000};
    if (dateRange.start) params.startDate = dateRange.start;
    if (dateRange.end)    params.endDate   = dateRange.end;
    if (report.filter)    Object.assign(params, report.filter);
    const res = await api.get(report.endpoint, { params });
    if (report.type === 'hmis') {
      const processors = {'hmis-opd':processOPDData,'hmis-ipd':processIPDData,'
      setReportData({ reportId:report.id, facilityName, dateRange, data:(proces
    } else {
      setReportData(res.data);
    }
  } catch (err) {
    const status = err.response?.status;
    if (status===403||status===401) setError('You do not have permission to acc
    else setError(err.response?.data?.message || 'Failed to generate report. Pl
  } finally { setLoading(false); }
};

```

```

const getRecordCount = () => {
  if (!reportData) return null;
  if (reportData.data?.totalVisits)      return reportData.data.totalVisits;
  if (reportData.data?.totalAdmissions) return reportData.data.totalAdmissions;
  if (reportData.data?.totalCases)       return reportData.data.totalCases;
  if (reportData.data?.invoices && Array.isArray(reportData.data.invoices)) ret
  if (Array.isArray(reportData.data))    return reportData.data.length;
  if (reportData.total !== undefined)     return reportData.total;
  if (reportData.count !== undefined)     return reportData.count;
  if (Array.isArray(reportData))          return reportData.length;
  return null;
};

```

```

const handleExport = (format) => {
  if (!reportData) return;
  if (format==='pdf') {

```

```

    buildPDF(reportData, selectedReport, dateRange, facilityName).save(`${selectedReport.name}.pdf`);
  } else if (format==='csv') {
    const rows=Array.isArray(reportData)?reportData:reportData?.data?.invoices?
    if(!rows.length) return;
    const keys=Object.keys(rows[0]).filter(k=>typeof rows[0][k]!='object');
    const csv=[keys.join(','),...rows.map(r=>keys.map(k=>{let v=r[k];if(typeof v==='object')
    const a=document.createElement('a');
    a.href=URL.createObjectURL(new Blob([csv],{type:'text/csv'}));
    a.download=`${selectedReport.name.replace(/\s+/g,'_')}_${new Date().toISOString()}.csv`;
    a.click();
  }
};

```

```

const isBillingStats = selectedReport?.id === 'billing-statistics';
const isIPDStats      = selectedReport?.id === 'ipd-statistics';
const isHMIS          = selectedReport?.type === 'hmis';
const recordCount     = getRecordCount();

```

```

return (
  <div className="min-h-screen bg-gray-50">
    <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-8">

      <div className="mb-8">
        <h1 className="text-2xl font-bold text-gray-900">Reports & Analytics</h1>
        <p className="text-sm text-gray-500 mt-1">Generate, view and export reports</p>
      </div>

      <div className="grid grid-cols-2 lg:grid-cols-4 gap-4 mb-8">
        <StatCard icon={Users} label="Active OPD Visits" value={fmt(stats.activeOpdVisits)}></StatCard>
        <StatCard icon={DollarSign} label="Total Collected" value={fmtTZS(stats.totalCollected)}></StatCard>
        <StatCard icon={Bed} label="Current Admissions" value={fmt(stats.currentAdmissions)}></StatCard>
        <StatCard icon={AlertCircle} label="Outstanding Balance" value={fmtTZS(stats.outstandingBalance)}></StatCard>
      </div>

      <div className="bg-white rounded-xl border border-gray-200 px-5 py-4 mb-6">
        <div className="flex-1 min-w-[220px]">
          <label className="block text-xs font-medium text-gray-500 mb-1">Facility Name</label>
          <input type="text" value={facilityName} onChange={e=>setFacilityName(e)}>
        </div>
      </div>

      <div>
        <label className="block text-xs font-medium text-gray-500 mb-1">From</label>
        <input type="date" value={dateRange.start} onChange={e=>setDateRange(e)}>
      </div>
    </div>
  </div>
)

```

```

        <label className="block text-xs font-medium text-gray-500 mb-1">To</l
        <input type="date" value={dateRange.end} onChange={e=>setDateRange(p=
            className="px-3 py-2 text-sm border border-gray-300 rounded-lg focu
    </div>
    {(dateRange.start||dateRange.end) && (
        <button onClick={()=>setDateRange({start:'',end:''})}
            className="flex items-center gap-1 px-3 py-2 text-sm text-gray-500
            <X className="w-3.5 h-3.5" /> Clear
        </button>
    )}
</div>

<div className="grid grid-cols-1 md:grid-cols-2 xl:grid-cols-3 gap-5 mb-8
    {reportCategories.map(cat=>{
        const Icon=cat.icon;
        return (
            <div key={cat.id} className="bg-white rounded-xl border border-gray
                <div className={`bg-gradient-to-r ${cat.color} px-4 py-3 flex ite
                    <div className="bg-white/20 rounded-lg p-1.5"><Icon className="
                        <h2 className="text-sm font-semibold text-white">{cat.title}</h
                    </div>
                <div className="divide-y divide-gray-100">
                    {cat.reports.map(report=>{
                        const isSelected=selectedReport?.id===report.id;
                        return (
                            <button key={report.id} onClick={()=>handleGenerateReport(r
                                className={`w-full text-left px-4 py-3 flex items-center
                                    <div>
                                        <div className={`text-sm font-medium ${isSelected?'text
                                            <div className="text-xs text-gray-400 mt-0.5">{report.d
                                        </div>
                                    <ChevronRight className={`w-4 h-4 flex-shrink-0 ml-2 ${is
                                </button>
                            );
                        })}
                    </div>
                </div>
            );
        )}
    </div>

    {loading && (
        <div className="bg-white rounded-xl border border-gray-200 p-10 flex it
            <RefreshCw className="w-5 h-5 text-blue-600 animate-spin" />
            <span className="text-gray-600 text-sm">Generating report...</span>
        </div>
    )}

```



```

{error && !loading && (
  <div className="bg-red-50 border border-red-200 rounded-xl p-4 mb-6 flex
    <AlertCircle className="w-5 h-5 text-red-500 flex-shrink-0 mt-0.5" />
    <div>
      <p className="text-sm font-medium text-red-800">Failed to generate
      <p className="text-sm text-red-600 mt-0.5">{error}</p>
    </div>
  </div>
)}

{reportData && !loading && (
  <div className="bg-white rounded-xl border border-gray-200 shadow-sm ov
    <div className="flex items-center justify-between px-5 py-4 border-b
      <div>
        <h3 className="text-base font-semibold text-gray-900">{selectedRe
        <p className="text-xs text-gray-400 mt-0.5">
          {recordCount!==null ? `${fmt(recordCount)} record${recordCount!
          {dateRange.start ? ` ? ${dateRange.start} -> ${dateRange.end} | `
          {' ? '}{new Date().toLocaleString()}
        </p>
      </div>
    </div>
    <div className="flex gap-2">
      <button onClick={()=>handleExport('pdf')}
        className="flex items-center gap-1.5 px-3 py-1.5 bg-red-600 tex
        <Download className="w-3.5 h-3.5" /> PDF
      </button>
      <button onClick={()=>handleExport('csv')}
        className="flex items-center gap-1.5 px-3 py-1.5 bg-emerald-600
        <Download className="w-3.5 h-3.5" /> CSV
      </button>
    </div>
  </div>
  <div className="p-5">
    {isHMIS
      ? <HMISSummary data={reportData} reportId={selectedReport.id} />
      : isBillingStats
      ? <BillingStatsSummary data={reportData} />
      : isIPDStats
      ? <IPDStatsSummary data={reportData} />
      : <ReportTable report={selectedReport} data={reportData} />
    }
  </div>
</div>
)}
</div>

```

```
        </div>
    );
};

export default Reports;
```